

VR-Based Human Feedback for Interactive Reinforcement Learning

Behrouz Ghamkhar

Abstract—This report presents a complete Interactive Reinforcement Learning (IRL) system for training a Pepper robot to perform contextually appropriate social greeting behaviours (DoNothing, Talk, Look, Wave, HandShake) in response to human cues. Three conditions are designed and evaluated: (1) an autonomous RL baseline using Proximal Policy Optimisation (PPO), (2) COACH fine-tuning with keyboard feedback, and (3) COACH fine-tuning with multimodal VR feedback. Conditions (2) and (3) independently fine-tune from the same pre-trained PPO checkpoint, allowing direct comparison of feedback modalities against each other and against the autonomous baseline. The PPO baseline uses a 12-dimensional observation space incorporating body-signal features (wrist height, wrist-to-core distance, body orientation, gaze direction), achieving a final training accuracy of 66.4% and a test accuracy of 74.14% after $\sim 150k$ training steps; a hyperparameter sweep across 10 configurations (30 runs total) identified learning rate ($\alpha = 0.001$) as the dominant factor. Because the limited human interaction budget is insufficient for PPO’s batch-gradient requirements, both fine-tuning conditions use COACH (COrrective ADvice Communicated by Humans), an algorithm that performs an immediate policy gradient update after every single feedback event. This targeted fine-tuning is designed to correct the complete Talk action suppression (0% action distribution) introduced by feature overlap with the Look condition. The keyboard COACH session ran for 164 feedback steps and produced a measurable policy shift: action distribution entropy rose from 1.39 to 1.79 bits and HandShake occurrences increased $6.85\times$. The VR condition—including controller buttons, head gestures (nod/shake), and voice commands fused via priority-based signal fusion—was fully implemented but could not be executed due to a physical body incompatibility between the NPC rig and the VR person controller during live sessions. Body-signal observations are normalised relative to each person’s standing height, ensuring scale-invariant feature representations that generalise across the NPC rig and real human participants of differing heights. Evaluation compares all conditions on final task accuracy, action distribution entropy, and HandShake engagement as a proxy for policy diversity improvement.

Index Terms—Interactive Reinforcement Learning, Social Robot Greeting, Multimodal VR Feedback.

I. Introduction

A. Motivation

Robots have the potential to become important partners in human environments, assisting with tasks at home or in complex collaborative work [1, 4]. However, enabling

robots to operate well in open-world environments—with unknown tasks, new objects, and dynamic human interactions—remains a central challenge. Robots often lack the common-sense contextual understanding that humans use effortlessly to generalise skills and interpret underspecified instructions.

A promising direction to address this challenge is Interactive Reinforcement Learning (IRL), which incorporates human feedback directly into the robot’s learning loop [3, 12]. By learning from evaluative or instructive feedback, IRL agents can adapt to complex, real-world constraints more effectively than autonomous RL agents that rely solely on engineered reward functions, while also improving sample efficiency and aligning behaviour with human preferences [12].

Despite its potential, a significant bottleneck in IRL is the interface through which feedback is delivered. Most existing systems rely on low-bandwidth, explicit channels such as keyboard presses or button clicks [10, 14]. These methods are unnatural for the human teacher and fail to capture the rich details of human communication—gesture, gaze, tone of voice—creating an expressiveness gap that limits the quality of information transferred from human to robot [13, 24].

At the same time, advances in consumer-grade Virtual Reality (VR) technology offer new opportunities for human-computer interaction. Modern VR headsets provide dense multimodal sensing: head pose, hand gestures, eye gaze, and voice input within a single, immersive platform [15, 26]. Despite this potential, VR and IRL have mostly developed in parallel. VR has been used for collecting demonstration data [8, 18] and, more recently, for preference-based learning [7], but its use for real-time scalar feedback within an IRL framework remains largely unexplored.

B. Problem Statement

The combination of the expressiveness gap in existing IRL interfaces, the limited exploitation of VR’s real-time sensing capabilities for feedback, and the unresolved challenge of multimodal signal fusion defines the core problem addressed in this work. Specifically:

- Current IRL feedback interfaces are narrow-bandwidth (typically binary key presses) and unnatural, limiting the quality of human supervision.
- VR headsets provide rich, synchronised multimodal signals (head pose, gaze, hand gestures, voice) that have not been systematically leveraged for real-time IRL.

*Submitted to the Department of Computer Science at Hochschule Bonn-Rhein-Sieg in partial fulfilment of the requirements for the degree of Master of Science in Autonomous Systems

[†]Supervised by Prof. Dr. Teena Chakkalayil Hassan (Hochschule Bonn-Rhein-Sieg) and Michal Stolarz (Hochschule Bonn-Rhein-Sieg)

[‡]Submitted in May 2026

- It is unclear which combinations of VR signals are most effective as reward proxies, or how to fuse them robustly in the presence of noise and conflicts.
- Standard on-policy RL algorithms (e.g., PPO) require large batches of environment interactions per gradient update, making them impractical when the human feedback budget is limited to a small number of interaction steps.

This work addresses these gaps through the following research questions:

- 1) Feasibility: Can multimodal feedback from a VR headset be effectively translated into a robust and informative reward signal for an RL agent in a greeting interaction task?
- 2) Performance Comparison: How does the learning efficiency and final performance of an agent fine-tuned with VR feedback compare to one trained with traditional keyboard input or an autonomous reward function?
- 3) User Experience: What are the qualitative differences in the teaching experience between a VR interface and a keyboard interface for providing IRL feedback?

C. Proposed Approach

We develop a proof-of-concept IRL system built around a social robot greeting task implemented in the Unity game engine. A Pepper robot agent must learn to select the contextually appropriate greeting behaviour—DoNothing, Talk, Look, Wave, or HandShake—in response to cues provided by an approaching human. Three conditions are designed:

- 1) Autonomous RL Baseline: The agent learns from scratch using PPO with a hand-crafted reward function (+1.0 for correct actions, -0.2 for incorrect actions, -0.0001 per-step penalty) for $\sim 150k$ steps. This establishes performance benchmarks and exposes the Talk action suppression that motivates the fine-tuning conditions.
- 2) Keyboard IRL: The best PPO checkpoint is loaded into the COACH agent. A human teacher provides scalar feedback (+1/-1) via keyboard arrow keys. COACH performs immediate per-event policy gradient updates suited to the limited human interaction budget; the session ran for 164 steps.
- 3) VR IRL: The same PPO checkpoint is independently loaded into a second COACH agent. A human teacher wearing a VR headset provides feedback through multiple modalities: controller buttons (explicit ± 1), head gestures (nod for positive, shake for negative), and voice commands (“good”/“bad”). This condition was fully implemented but could not be executed due to a physical body incompatibility between the NPC rig and the VR person controller.

Conditions (2) and (3) both branch from the same PPO baseline checkpoint independently—neither depends on the other. This allows a direct comparison of feedback

modality (keyboard vs VR) against each other and against the autonomous baseline. Body-signal observations are normalised relative to each person’s standing height, ensuring scale-invariant features across the NPC training rig and VR participants of differing heights.

D. Contributions

This report makes the following specific contributions:

- 1) Complete System Architecture: Integration of Unity ML-Agents with PyTorch PPO and COACH, including a 12-dimensional observation space with body-signal features (wrist height, wrist-to-core distance, body orientation, gaze direction) that replace explicit task IDs.
- 2) Hyperparameter Characterisation: Systematic sweep of 10 configurations varying learning rate (10^{-4} to 10^{-3}), network capacity (64, 128, 256 units), entropy schedule, rollout length, and PPO clip parameter, providing guidance for future social greeting tasks.
- 3) Performance Benchmarking: Quantitative results establishing final training accuracy (66.4%), test accuracy (74.14%), sample efficiency ($\sim 9k$ steps to 80% accuracy), and policy stability ($\sigma = 0.72$) for the autonomous condition as reference values for IRL comparison.
- 4) COACH Fine-Tuning Pipeline: Implementation of COACH as a targeted fine-tuning mechanism that corrects the Talk action suppression (0% distribution) introduced by body-signal feature overlap with the Look condition, using only 164 human feedback events loaded from the pre-trained PPO checkpoint.
- 5) Body-Relative Observation Normalisation: A scale-invariant normalisation scheme in which wrist height and wrist-to-core distance are expressed as fractions of the person’s own standing height, enabling consistent feature representations across NPC training and live VR human participants.
- 6) Multimodal VR Feedback Interface: A priority-based fusion of controller buttons, head gestures, and voice commands into a scalar reward signal, with per-modality reward magnitudes and timeout gating to prevent signal duplication.
- 7) Reproducible Configuration: Complete YAML configuration files (config1.yaml–config10.yaml and coach_config.yaml) and training code for replication and extension to further IRL conditions.

E. Report Structure

Section II reviews related work in IRL, VR-based robot learning, and avatar motion estimation. Section III introduces background concepts in reinforcement learning, PPO, IRL, and the COACH algorithm. Section IV describes the complete system design, including the training conditions, observation/action spaces, reward function, body-relative normalisation, and COACH fine-tuning methodology. Section V presents experimental results from the hyperparameter sweep, definitive PPO run,

and keyboard COACH fine-tuning evaluation. Section VI summarises findings and discusses implications for VR-based IRL research.

II. Related Work

Our work sits at the intersection of IRL, HRI, and VR interfaces. We review key contributions across these domains.

A. Interactive Reinforcement Learning (IRL)

The foundation of this review is based on important papers that shaped the IRL field. Key work by Bradley Knox and Stone [3] introduced TAMER, showing that human evaluative feedback could train agents much faster than autonomous RL. This was extended into social domains by Lin et al. [13] and Lin et al. [12] who provide a comprehensive review of IRL from human social feedback, distinguishing between evaluative feedback (e.g., rewards or punishments that signal approval or disapproval) and instructive feedback (e.g., advice, corrections, or demonstrations that guide the learner toward desired behaviors).

A number of IRL frameworks have been proposed to integrate human feedback in different ways. COACH uses human feedback directly to calculate the policy gradient in an actor-critic algorithm. This allows for more flexible guidance, particularly in continuous action spaces, as it enables the human teacher to provide feedback on the appropriateness of specific actions in given states, rather than just binary reward signals [12]. This approach supports real-time corrective feedback during learning, allowing the agent to adjust its policy immediately based on human input.

Early work by Sridharan [22] explored augmented reinforcement learning for interaction with non-experts. A significant challenge in IRL is the quality of human input; Loftin et al. [14] investigated using implicit human feedback strategies, while Faulkner et al. [10] and Kessler Faulkner and Thomaz [11] developed specific algorithms for learning from inaccurate or imperfect teachers. The field has explored various feedback modalities, including haptics [6] and even bio-signals, such as the use of EEG-based implicit human feedback to accelerate learning [27].

B. VR for Data Collection and Demonstration

The use of VR as a powerful tool for collecting human data for robot learning is well-established. A primary application has been in Learning from Demonstration (LfD) and robot programming, where methods range from multi-resolution corrective demonstration [17] and learning responsive behavior by imitation [2] to Bayesian fusion of multimodal advice [24]. Christen et al. [5] used motion capture data from human-human interactions to guide the reward function for training dexterous HRI policies. Mosbach et al. [18] used GPU-based simulation and high-quality demonstrations, often collected in VR, to accelerate manipulation learning. More recently, Dogangun et al.

[8] developed RAMPA, a system using Augmented Reality for machine programming by demonstration, highlighting the trend towards immersive technologies for intuitive teaching. This approach is also used in neurorehabilitation, as shown by Lydakis et al. [16], who developed a learning-based agent for home therapy using EMG and avatar-based feedback.

C. VR for Preference and Feedback

While VR for demonstration is mature, its use for providing evaluative feedback is an emerging area. The most directly related work is by de Heuvel et al. [7], who investigated the impact of VR and 2D interfaces on human feedback in preference-based robot learning. Their work focuses on comparing interfaces for offline, comparative feedback. Our project aims to extend this line of inquiry into the domain of real-time, in-the-loop scalar feedback, which is a distinct and unexplored challenge.

D. Avatar and Motion Control in VR

A critical technical challenge is interpreting the sparse sensor data from a VR headset into a meaningful representation of human state. This is an area of intense research in computer vision and graphics. Recent breakthroughs have employed Reinforcement Learning to create physically plausible avatars. Winkler et al. [26] presented QuestSim, an RL framework that takes sparse signals from a Head-Mounted Display and two controllers to simulate full-body motion. This work was advanced by Luo et al. [15], who developed a method for a real-time simulated avatar from head-mounted sensors, a technology crucial for providing the context-aware feedback our system intends to use. Further foundational work includes that of Ye et al. [28] on generating physically realistic full-body motion for VR users.

E. Other Relevant Works

Other relevant works include the integration of feedback in latent space for skill learning [19] and the use of fairness-sensitive policy gradients to reduce bias in robotic assistance [29]. The method of using adversarial motion priors as substitutes for complex reward functions [9] is also relevant for shaping agent behavior without manually engineering rewards.

F. Limitation and Deficits in the State of the Art

The current state of the art reveals several significant limitations that our project aims to address:

- 1) Expressiveness Gap in IRL: Existing IRL methods predominantly rely on low-dimensional, often binary feedback mechanisms such as keyboard presses or button clicks [10, 14]. These interfaces fail to capture the rich, nuanced feedback that humans can naturally provide through gestures, gaze, and body language, creating an expressiveness bottleneck in human-robot communication.

- 2) Underutilization of VR’s Potential for Real-time Feedback: While VR has been successfully used for collecting demonstration data [8, 18] and more recently for preference elicitation [7], its potential for capturing real-time, in-the-loop scalar feedback within IRL frameworks remains largely unexplored. The multimodal sensing capabilities of modern VR headsets offer distinct advantages over camera-based solutions for IRL applications: inherent 3D spatial awareness with precise head and hand tracking without calibration drift, and robust performance that is unaffected by lighting conditions, occlusions, or camera viewpoint limitations. Additionally, multimodal sensing combining pose, gaze, voice, and controller inputs in a single synchronized platform. These capabilities have not been fully leveraged for interactive machine learning.
- 3) Lack of Multimodal Signal Fusion for Feedback: While VR provides a multitude of signals (gaze, pose, voice), no existing work in RL systematically investigates which combinations are most effective for providing feedback or how to fuse them robustly for simple HRI tasks.

This project directly addresses these deficits by developing a novel VR-based IRL framework that uses the expressiveness of immersive interfaces and incorporates multimodal signal fusion for a greeting interaction task.

III. Background

A. Reinforcement Learning

Reinforcement Learning (RL) is a machine learning paradigm in which an autonomous agent learns to make sequential decisions by interacting with an environment. The core objective is to discover a policy—a mapping from environment states to actions—that maximises a cumulative numerical reward signal [12]. This interaction is formally modelled as a Markov Decision Process (MDP), defined by the tuple (S, A, P, R, γ) , where S is the set of states, A is the set of actions, $P(s_{t+1} | s_t, a_t)$ is the state-transition probability, $R(s_t, a_t, s_{t+1})$ is the reward function, and $\gamma \in [0, 1]$ is a discount factor weighting immediate versus future rewards.

The agent’s goal is to learn an optimal policy $\pi^*(a | s)$ that maximises the expected cumulative discounted return $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$. The state-value function $V^\pi(s)$ and the action-value function $Q^\pi(s, a)$ estimate the expected return from state s under policy π , and after taking action a in state s and thereafter following π , respectively.

B. Proximal Policy Optimisation

Proximal Policy Optimisation (PPO) [21] is a policy gradient algorithm that achieves stable updates by clipping the probability ratio between old and new policies. The PPO objective function is:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (1)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ is the probability ratio, $\hat{A}_t$ is the advantage estimate, and ϵ is the clip parameter (set to $\epsilon = 0.2$ in our experiments). The clipping mechanism prevents destructively large policy updates, improving stability compared to vanilla policy gradient methods.

The advantage is estimated using Generalised Advantage Estimation (GAE) [20]:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l} \quad (2)$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ is the TD error and λ is the GAE parameter (set to $\lambda = 0.95$ in our experiments), controlling the bias-variance trade-off in advantage estimation.

The total loss combines policy, value function, and entropy terms:

$$L_t^{\text{total}} = L_t^{\text{CLIP}} - c_1 L_t^{\text{VF}} + c_2 S[\pi_\theta](s_t) \quad (3)$$

where $c_1 = 0.5$ (value loss coefficient), c_2 is the entropy coefficient (annealed from 0.1 to 0.02 over training), and S is the entropy bonus that encourages exploration by penalising deterministic policies [25].

C. Interactive Reinforcement Learning

IRL extends the standard RL framework by replacing or augmenting the engineered reward signal R with feedback from a human teacher. Formally, at each timestep t the agent receives a human-provided signal $H_t \in \mathbb{R}$ (e.g., +1 for approval, −1 for disapproval) that is added to or substitutes for the environmental reward:

$$r_t^{\text{IRL}} = \lambda_e R(s_t, a_t) + \lambda_h H_t, \quad (4)$$

where λ_e and λ_h are weighting coefficients. This formulation, sometimes called reward shaping with human feedback, allows the teacher to steer the agent’s learning without needing to specify a complete reward function [10]. In our implementation the two conditions are architecturally exclusive rather than blended at runtime: in the autonomous baseline $\lambda_h = 0$ (no human signal), and in the pure IRL conditions (keyboard and VR) $\lambda_e = 0$ (the autonomous reward function is bypassed entirely and all reward delivery is handled by the active IReward-Provider). Equation 4 therefore serves as a conceptual framing of the design space rather than a runtime weighted sum.

D. COACH: Corrective Advice Communicated by Humans

COACH [12] is an IRL algorithm designed specifically for the low-sample regime in which on-policy batch methods such as PPO are infeasible. Rather than accumulating rollout buffers for batch updates, COACH performs a single policy gradient step immediately after every non-trivial human feedback signal:

$$\mathcal{L}_{\text{COACH}}(\theta) = -f_t \log \pi_\theta(a_t | s_t) - c_2 \mathcal{H}[\pi_\theta(\cdot | s_t)] \quad (5)$$

where $f_t \in \mathbb{R}$ is the scalar feedback at step t , $\pi_\theta(a_t | s_t)$ is the policy probability of the taken action, and $c_2 \mathcal{H}[\pi_\theta]$ is an entropy bonus that maintains exploration during fine-tuning. Positive feedback ($f_t > 0$) reinforces the chosen action by increasing its log-probability; negative feedback ($f_t < 0$) suppresses it and implicitly redistributes probability mass to alternative actions.

This formulation has three properties that make it well-suited to VR fine-tuning. First, it requires no rollout buffer, value function, or advantage estimation—a single (s_t, a_t, f_t) triple is sufficient for a gradient update. Second, updates are triggered only when $|f_t| > \epsilon_{\text{threshold}}$ (set to 0.05 in our implementation), so the small per-step penalty (-0.0001) is silently ignored. Third, the policy gradient is the same mathematical object as the actor update in a standard actor-critic, so a network pre-trained by PPO can be loaded directly into the COACH agent without architectural changes.

1) Why PPO Cannot Learn from Sparse Human Feedback: PPO requires batch sizes of 256–2048 steps per gradient update for stable training; with ≈ 200 VR interaction steps available in total, at most one or two PPO updates are possible, producing gradient estimates far too noisy to shift a converged policy. The situation is compounded by action sparsity: the target action (Talk) appears in only $\sim 7\%$ of natural encounters, yielding fewer than 15 Talk-state feedback events in a 200-step VR session—insufficient for PPO’s batch normalisation and clipping mechanisms to function correctly. COACH’s immediate, per-event update mechanism circumvents both limitations, making it the appropriate algorithm for this fine-tuning stage.

E. Multimodal Signal Fusion for Reward

When the feedback channel is a VR headset, the teacher’s intent may be expressed through multiple simultaneous modalities: explicit controller button presses (low latency, high reliability), implicit head gestures (nod for positive, shake for negative, medium latency), and spoken voice commands (higher latency, variable reliability). Each modality carries different characteristics.

A fusion function $\Phi(\cdot)$ maps the vector of active modality readings $\mathbf{m}_t = (m_t^{\text{btn}}, m_t^{\text{head}}, m_t^{\text{voice}})$ to a scalar reward:

$$H_t = \Phi(\mathbf{m}_t) \in [-1, +1]. \quad (6)$$

In practice each modality produces a fixed scalar magnitude rather than a binary value: controller buttons emit ± 1.0 , voice commands emit ± 0.75 , and head gestures (nod/shake) emit $+0.5/-0.5$. Designing Φ to be robust to noise and modality conflicts—e.g., a head nod occurring simultaneously with a negative button press—is one of the central engineering challenges addressed in Sec. IV. In our implementation, we adopt a simple priority-based fusion: explicit button inputs override other modalities

when present, followed by voice, followed by head gestures, with a cooldown window to prevent repeated signals from a single gesture.

F. The Greeting Task as an MDP

Our greeting interaction scenario is formally defined as an MDP with the following components:

State Space S : 12-dimensional continuous vector as detailed in Table I. This includes Pepper’s internal state (5-dim one-hot), normalised distance to the human (1-dim), handshake flags (2-dim binary), and body-signal features (4-dim continuous).

Action Space A : Five discrete actions: DoNothing (0), Talk (1), Look (2), Wave (3), HandShake (4).

Transition Function P : Deterministic transitions based on the selected action and the NPC’s current task. The NPC task selection follows a uniform random distribution over the task pool.

Reward Function R : Defined in Table III. For the autonomous baseline, rewards are computed by CommunicationManager based on task-action matching. For IRL conditions, the human teacher provides the reward signal.

Discount Factor γ : Set to $\gamma = 0.99$, encouraging the agent to consider future rewards while maintaining sensitivity to immediate feedback.

IV. Methodology

This section describes the complete experimental setup across all three training conditions. The autonomous PPO baseline (Section IV-B) is trained first to establish performance benchmarks and identify failure modes. The two COACH fine-tuning conditions (keyboard and VR) then independently load the same best-performing PPO checkpoint, allowing direct comparison of feedback modalities without any dependency between them.

A. System Architecture

The system architecture comprises three primary components: the Unity simulation environment (providing the greeting task and agent controller), the Python training framework (implementing PPO and COACH), and the ML-Agents communication protocol (enabling real-time interaction between Unity and Python). Figure 1 illustrates the data flow.

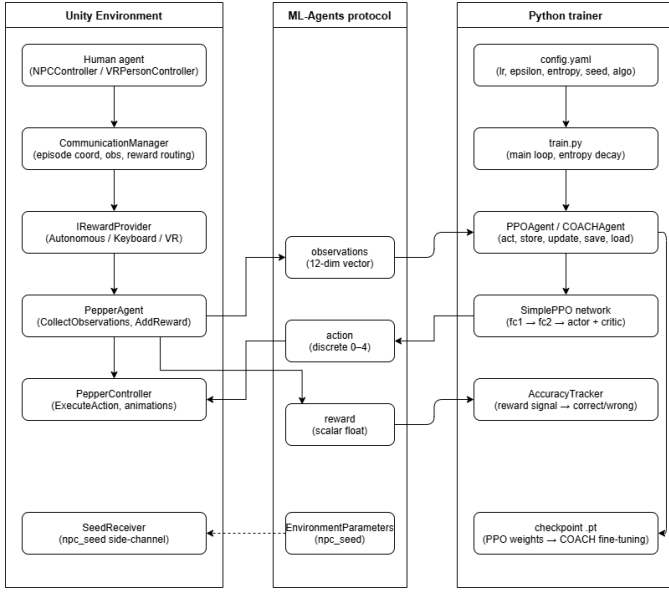


Fig. 1: System Architecture Overview. The Unity environment (left) hosts the CommunicationManager, human agent (NPC or VR participant), IRewardProvider, PepperAgent, and PepperController. The ML-Agents protocol (centre) carries the 12-dimensional observation vector, discrete action, and scalar reward between Unity and Python. The Python trainer (right) runs train.py with the PPOAgent or COACHAgent, both backed by the shared SimplePPO network, and saves checkpoints that are reloaded for COACH fine-tuning.

The Unity environment implements the CommunicationManager as the central coordinator, responsible for: (1) forwarding the agent’s chosen action to PepperController, (2) computing rewards based on task-action matching (autonomous mode) or routing human feedback signals (IRL mode), (3) resetting the scene at episode boundaries, and (4) exposing body-signal observations for the RL policy. The PepperAgent class inherits from ML-Agents’ Agent class and implements observation collection, action execution, and episode termination logic.

B. Training Conditions

Three conditions are evaluated. The two COACH conditions branch independently from the same PPO checkpoint and do not depend on each other:

- 1) **Autonomous PPO Baseline:** The agent trains from scratch using PPO for $\sim 150k$ steps against a hand-crafted reward function (Table III). An NPC character presents tasks; no human is involved. Ten configurations are swept; the best-performing checkpoint (config3, lr=0.001) is saved and used as the starting point for both fine-tuning conditions.
- 2) **COACH Keyboard Fine-Tuning:** The PPO checkpoint is loaded into a fresh COACH agent. A human teacher provides scalar feedback (+1/−1) via keyboard arrow keys. The shared SimplePPO network architecture allows direct weight transfer.

COACH updates the policy immediately on every non-trivial feedback event, targeting correction of the Talk action bias identified in the baseline.

- 3) **COACH VR Fine-Tuning:** The same PPO checkpoint is independently loaded into a second fresh COACH agent. Feedback is delivered multimodally via a VR headset (controller buttons, head gestures, voice commands). The Unity environment uses the VR scene instead of the Baseline scene; the Python training loop is otherwise identical to condition (2).

Both COACH conditions use the same train.py loop as PPO—the algorithm is selected via the algorithm: coach field in the configuration file, which routes instantiation to COACHAgent through the ALGORITHMS registry. The RESUME_FROM field points to the PPO checkpoint, causing the agent to load PPO weights while resetting the optimiser state to the COACH learning rate. The two COACH runs are executed separately and produce independent fine-tuned checkpoints, both evaluated against the same autonomous baseline.

C. Environment Specification

1) **Greeting Task Definition:** The greeting interaction task was implemented in the Unity game engine. A Pepper robot agent occupies a fixed position in the arena; a human agent (NPC during training, VR participant during IRL) approaches from varying distances and presents one of four situational cues, each associated with a socially appropriate greeting response. In autonomous and keyboard modes, the NPC’s behaviour cycle is governed by NPCController, which selects tasks uniformly at random from its task pool and navigates to target positions via Unity’s NavMesh system. In VR mode, the human participant manually selects and performs tasks via controller input, replacing the NPC entirely. The episode terminates upon a successful handshake completion, after reaching the maximum step limit (2000 steps per episode), or after a wall-clock timeout of 60 seconds.

2) **Observation Space (12 Dimensions):** The agent receives a 12-dimensional observation vector at each decision step, as detailed in Table I. The body-signal features replace an earlier task one-hot encoding, allowing the agent to infer the human’s intent from posture rather than requiring explicit task labels. Critically, features [8] and [9] are normalised relative to the person’s own standing height (see Section IV-D), making them scale-invariant across the NPC rig and VR participants of differing heights.

3) **Action Space (5 Discrete Actions):** The agent operates in a discrete action space of five actions, as defined in PepperController.AgentAction. Actions are selected at each decision step by the policy network and forwarded to PepperController, which triggers the corresponding animation and state transition.

TABLE I: Complete Observation Space Definition

Idx	Feature	Range	Meaning
0–4	PepperState	one-hot (5)	Idle / Looking / Waving / Handshaking / PerformingAction
5	Distance to human	[0, 1]	Normalised by 10m; 0=at Pepper, 1=far away
6	HandshakeInProgress	{0, 1}	1 when a handshake sequence is active
7	CanHandshake	{0, 1}	1 when handshake cooldown (3s) has elapsed
8	WristHeight	[0, 1]	Wrist height above floor / person standing height
9	WristToCoreDistance	[0, 1]	Wrist-to-hip distance / person standing height
10	BodyOrientation	[−1, 1]	Body facing Pepper (+1) vs away (−1)
11	GazeDirection	[−1, 1]	Head/gaze towards Pepper (+1) vs away (−1)

TABLE III: Complete Reward Mapping (Autonomous Baseline)

Task ID	NPC Situation	Correct Action	Correct Reward	Wrong Action Reward
2	NPC requests handshake	HandShake	+1.0	−0.2
7	NPC requests wave	Wave	+1.0	−0.2
6	NPC requests talk	Talk	+1.0	−0.2
Other / nearby	NPC ≤ 5 m from Pepper	Look	+1.0	−0.2
Other / far	NPC > 5 m from Pepper	DoNothing	+0.001	−0.2
Any	Per step (any)	— (penalty)	−0.0001	Always applied

TABLE II: Action Space Definition

Action ID	Name	Appropriate Context
0	DoNothing	NPC > 5 m away (far)
1	Talk	Task ID 6 (talk request)
2	Look	NPC ≤ 5 m away (near)
3	Wave	Task ID 7 (wave request)
4	HandShake	Task ID 2 (handshake request)

4) Reward Function Definition: The autonomous reward signal is computed by `CommunicationManager.EvaluateReward()`. A reward gate ensures that only the first action performed per task event receives a reward, preventing reward hacking through action repetition. Table III shows the complete mapping.

The per-step penalty of -0.0001 encourages timely responses. In VR and keyboard IRL modes, the autonomous reward function is replaced entirely by the human feedback signal; `EvaluateReward()` returns immediately without assigning any reward, and all reward delivery is handled by the active `IRewardProvider`.

D. Body-Relative Observation Normalisation

1) Motivation: Initial training used fixed normalisation constants for body-signal features: wrist height was divided by a global maximum of 2.5 m, and wrist-to-core distance by 1.5 m. These constants were calibrated for the NPC rig. When a live human replaces the NPC in VR mode, two problems arise: (1) the human’s absolute wrist height for the same proportional gesture differs with their height, and (2) the NPC’s hip bone (used as the

core reference) has no direct equivalent in the XR rig—a static empty `GameObject` placed at the XR Origin root approximates the core at floor level (≈ 0 m) rather than hip level (≈ 1 m), causing `WristToCoreDistance` to be systematically overestimated for VR participants.

2) Body-Relative Scheme: Both features are now normalised by the person’s own standing height, computed dynamically as:

$$h_{\text{person}} = \max(0.1, y_{\text{head}} - y_{\text{floor}}) \quad (7)$$

where y_{head} is the world-space y -coordinate of the head bone (NPC) or Main Camera (VR), and y_{floor} is the configured floor reference (default: 0). The revised features become:

$$\text{WristHeight} = \frac{y_{\text{wrist}} - y_{\text{floor}}}{h_{\text{person}}} + \eta \quad (8)$$

$$\text{WristToCoreDistance} = \frac{\|\mathbf{p}_{\text{wrist}} - \mathbf{p}_{\text{hip}}\|}{h_{\text{person}}} + \eta \quad (9)$$

where $\eta \sim \mathcal{U}(-\nu, +\nu)$ is additive noise ($\nu = 0.05$) and $\mathbf{p}_{\text{hip}}$ is an approximate hip position derived as:

$$\mathbf{p}_{\text{hip}} = (x_{\text{head}}, y_{\text{floor}} + 0.55 h_{\text{person}}, z_{\text{head}}) \quad (10)$$

The constant 0.55 was chosen, because hip height is seems to be approximately 55% of standing height for an adult. This approximation is applied identically for NPC and VR participants, requiring only the head transform to be assigned in the Unity Inspector for each scene—no separate hip bone assignment is needed.

3) Properties: Under this scheme, two people performing the same proportional gesture—regardless of absolute height—produce near-identical observation values, with remaining variation absorbed by the ± 0.05 noise already present during training. The normalisation is also robust to VR headset floor-level calibration drift, since both y_{wrist} and y_{head} share the same world-space origin.

E. Observation Robustness: Temporal Sampling and Input Noise

Two additional measures improve the reliability of body-signal observations and reduce the sim-to-real gap when transitioning from NPC-based training to a live human teacher in the VR condition.

Delayed observation sampling. Observations are collected after a per-task delay (`observationDelay`, default: 1.0s) so that observations are sampled once the animation has reached a recognisable mid-gesture state. The delay is jittered by $\pm \text{observationDelayNoise}$ (default: 0.2s) each episode to prevent the agent from overfitting to a single animation frame.

Observation value noise. Small additive noise $\eta \sim \mathcal{U}(-\nu, +\nu)$ ($\nu = 0.05$) is applied to `WristHeight` and `WristToCoreDistance` before they enter the observation vector, broadening the training distribution in a manner analogous to domain randomisation [23].

Together, these measures ensure the agent trains on a distribution of body-signal values rather than a deterministic trajectory, improving robustness to the human motion variability expected in VR deployment.

F. Training Algorithm - Stage 1: PPO

Training used Proximal Policy Optimisation (PPO) as implemented in `algorithms/ppo.py`. The policy and value networks share a fully-connected architecture (SimplePPO) with a shared trunk of two hidden layers, separate actor and critic heads, and orthogonal weight initialisation with gain $\sqrt{2}$. The actor head outputs logits over 5 discrete actions; the critic head outputs a scalar value estimate.

Key hyperparameters for the baseline configuration (`config1.yaml`) are shown in Table IV.

G. Experimental Configurations — Hyperparameter Sweep

To characterise the task and identify optimal settings, a hyperparameter sweep was conducted across 10 configurations, as shown in Table V. Each configuration varies exactly one hyperparameter group relative to the baseline to ensure interpretable comparisons.

To ensure robust results, each configuration was run with 3 random seeds (total 30 training runs). The parallel training runner in `train.py` executed configurations simultaneously on separate Unity instances with unique worker ports, respecting the `MAX_PARALLEL` limit.

H. Training Algorithm — Stage 2: COACH Fine-Tuning

1) Motivation: Analysis of the PPO baseline revealed a persistent Talk action bias: despite achieving 76.0% overall accuracy, the Talk action was selected in only 7.2% of decisions (compared to Look at 38.3%), leading to reduced accuracy specifically on task ID 6. This arises because the body-signal features for a talk request (neutral posture, facing Pepper at close range) overlap substantially with the Look condition, causing the policy to favour the more frequently rewarded Look action.

PPO cannot correct this bias through VR human feedback alone: the algorithm requires batches of 256–2048 steps per gradient update, yet the total available VR feedback budget is ~ 150 –300 interaction steps. At a natural task distribution, only ~ 7 –22 of those steps involve a Talk-state event—far too few for PPO’s batch normalisation and advantage estimation to produce a useful gradient. COACH (Section III-D) is chosen instead, as its immediate per-event update requires no buffer and directly weights the policy gradient by the human’s scalar feedback signal.

2) Implementation: The COACH agent (`algorithms/coach.py`) shares the SimplePPO network architecture with the PPO agent, preserving both the actor and critic heads for weight-loading compatibility. Only the actor head participates in COACH updates; the critic output is discarded. The update rule applied after each non-trivial feedback event ($|r| > 0.05$) is:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} [-f_t \log \pi_{\theta}(a_t | s_t) - c_2 \mathcal{H}[\pi_{\theta}(\cdot | s_t)]] \quad (11)$$

where $\alpha = 0.0005$ (higher than PPO’s 0.0003 to compensate for the small number of updates), $c_2 = 0.05$ (reset from PPO’s decayed value to maintain exploration), and gradients are clipped to max norm 0.5.

The COACHAgent interface is fully compatible with `train.py`’s training loop: `act()` returns a fake value of 0.0 to satisfy the loop’s three-tuple unpack; `store_transition()` triggers `_coach_update()` internally when feedback is non-trivial; `update()` and `get_value()` are no-ops. This means the entire PPO training loop in `train.py` runs unchanged for COACH sessions—only the `algorithm: coach` field in the configuration file and the `RESUME_FROM` checkpoint path need to be set.

3) COACH Configuration: Key COACH fine-tuning hyperparameters are shown in Table VI. The `hidden_units` value must match the PPO checkpoint exactly (128) for weight loading to succeed.

TABLE IV: Baseline Hyperparameters (config1.yaml)

Parameter	Value	Notes
learning_rate	0.0003	Adam optimiser
epsilon (clip)	0.2	PPO clip parameter
num_epoch	4	PPO update epochs per rollout
num_mini_batch	4	Mini-batches per epoch
value_loss_coef	0.5	Critic loss weight
entropy_start	0.1	Initial exploration bonus
entropy_end	0.02	Final entropy after full training
lambda (GAE λ)	0.95	Advantage estimation decay
gamma	0.99	Discount factor
hidden_units	128	Neurons per hidden layer
rollout_steps	256	Steps collected before each update
max_steps	150,000	Total training steps
max_grad_norm	0.5	Gradient clipping threshold

TABLE V: Hyperparameter Sweep Configurations

Config	What Changes	Changed Value	Baseline	Hypothesis
config1	Baseline (reference)	—	—	Establishes learning floor/ceiling
config2	Lower learning rate	lr = 0.0001	0.0003	More stable but slower convergence
config3	Higher learning rate	lr = 0.001	0.0003	Faster early learning, possible instability
config4	Larger network	hidden = 256	128	More capacity for body-signal features
config5	Smaller network	hidden = 64	128	Test if task is simple enough
config6	High entropy	ent_start=0.3, end=0.05	0.1/0.02	More exploration of less common actions
config7	Low entropy	ent_start=0.05, end=0.005	0.1/0.02	Faster exploitation, risk of premature convergence
config8	Longer rollout	rollout = 512	256	Smoother gradient estimates
config9	Shorter rollout	rollout = 128	256	More frequent updates, higher variance
config10	Tighter clip	epsilon = 0.1	0.2	More conservative policy updates

TABLE VI: COACH Fine-Tuning Hyperparameters (coach_config.yaml)

Parameter	Value	Notes
learning_rate	0.0002	Higher than PPO; sparse updates
entropy_start	0.5	Reset from PPO's decayed value
entropy_end	0.2	Gentle decay over session
hidden_units	128	Must match PPO checkpoint
max_steps	164	VR interaction steps
max_grad_norm	0.5	Same as PPO
time_scale	1	Real-time for live human feedback
algorithm	coach	Selects COACHAgent

Algorithm 1 COACH Fine-Tuning Loop

Require: Pre-trained policy π_θ (loaded from PPO checkpoint), COACH hyperparameters

- 1: Reset optimiser state (fresh Adam at COACH learning rate)
- 2: for each VR interaction step t do
- 3: Observe s_t from Unity environment
- 4: Sample $a_t \sim \pi_\theta(\cdot | s_t)$
- 5: Execute a_t ; receive human feedback f_t
- 6: if $|f_t| > 0.05$ then ▷ Non-trivial feedback only
- 7: Compute $\mathcal{L} = -f_t \log \pi_\theta(a_t | s_t) - c_2 \mathcal{H}[\pi_\theta(\cdot | s_t)]$
- 8: Update θ via Adam with gradient clipping
- 9: Save fine-tuned checkpoint

4) COACH Training Loop:

5) Expected Convergence: Because the system is event-driven (one NPC task or VR gesture = one agent decision = one feedback signal), every step carries a complete (s_t, a_t, f_t) tuple. Prior to the VR session, the NPC task scheduler is configured to over-sample Talk-state events ($\sim 50\%$ of encounters) so that the human's feedback

budget is concentrated on the problematic action. Under this setting, 150–200 VR interaction steps are expected to yield 75–100 Talk-specific feedback events—sufficient for COACH to meaningfully shift the policy’s logits in Talk states without degrading performance on the remaining actions, which continue to receive sparse or no corrective feedback.

I. Training Implementation Details

The training loop follows the standard PPO workflow for Stage 1 and the COACH immediate-update pattern for Stages 2a and 2b:

Algorithm 2 PPO Training Loop (Stage 1)

Require: Initial policy π_θ , value function V_ϕ , hyperparameters

- 1: for each iteration do
 - 2: Collect $\{s_t, a_t, r_t, s_{t+1}\}$ for T steps using π_θ
 - 3: Compute advantages $\hat{A}_t$ using $\text{GAE}(\gamma, \lambda)$
 - 4: for each PPO epoch do
 - 5: Sample mini-batches from collected data
 - 6: Compute policy loss $L^{CLIP}(\theta)$
 - 7: Compute value loss $L^{VF}(\phi)$
 - 8: Update θ, ϕ using Adam optimiser
 - 9: Update entropy coefficient c_2 (linear schedule)
-

Key implementation features include:

- Reward gate: Only the first action per task event receives a reward, preventing action spamming.
- Entropy annealing: Linear schedule from entropy_start to entropy_end over total steps; applies to both PPO and COACH.
- Handshake cooldown: 3-second cooldown prevents repeated handshake attempts.
- Action logging: Action distribution recorded periodically to detect action collapse or bias correction progress.
- Checkpoint compatibility: PPO and COACH use the same torch.save dictionary format, enabling seamless weight transfer between stages.

J. Metrics and Evaluation Protocol

The following metrics are recorded during training and used for cross-condition comparison:

The evaluation protocol defines four criteria for assessing all conditions:

- 1) Task Accuracy: Percentage of correct actions across all task contexts.
- 2) Talk Task Accuracy: Per-task accuracy specifically for task ID 6, the primary COACH correction target.
- 3) Sample Efficiency: Number of training steps (PPO) or feedback events (COACH) required to reach 80% overall accuracy.
- 4) Policy Stability: Standard deviation of rolling average reward post-convergence; action distribution entropy.

K. Reproducibility and Configuration Files

All experiments are reproducible via the provided configuration files. The file structure is:

```
configs/
  config1.yaml      # PPO Baseline
  config2.yaml      # Lower learning rate
  config3.yaml      # Higher learning rate (best)
  config4.yaml      # Larger network (256 units)
  config5.yaml      # Smaller network (64 units)
  config6.yaml      # High entropy schedule
  config7.yaml      # Low entropy schedule
  config8.yaml      # Longer rollout (512 steps)
  config9.yaml      # Shorter rollout (128 steps)
  config10.yaml     # Tighter clip (epsilon=0.1)
configs/Finetune/
  coach_config.yaml # COACH fine-tuning (keyboard + VR)
```

Each YAML file specifies the complete hyperparameter set for a given condition. The train.py script discovers all YAML files in the target directory and executes them in parallel (PPO) or sequentially (COACH/VR) using Python’s multiprocessing.Pool.

V. Evaluation

This section presents experimental results across all three training conditions. Section V-A covers the hyperparameter sweep and Section V-B the definitive autonomous PPO baseline. Section V-F presents COACH fine-tuning results for the keyboard condition. The VR COACH condition was implemented and tested in simulation but a full fine-tuning session could not be completed due to physical incompatibility between the NPC rig and the VR person controller’s collision body, which prevented stable interaction during live VR sessions. All PPO results are averaged over 3 random seeds unless otherwise specified.

A. Hyperparameter Sweep Results

The hyperparameter sweep revealed significant differences across configurations. Table VIII summarises the performance of each configuration; Figure 2 shows the full smoothed reward and accuracy trajectories for all 31 runs.

1) Learning Rate Effects (Config2, Config3): Reducing the learning rate to 0.0001 (config2) yielded marginally improved stability (post-convergence $\sigma = 0.57$ vs baseline $\sigma = 0.76$), but at a severe cost: config2 required 33k steps to reach 80% accuracy versus 12k for the baseline and achieved markedly lower final accuracy (50.9% vs 63.0%).

Increasing the learning rate to 0.001 (config3) produced the best overall outcome in the sweep: the highest mean final accuracy (65.2%), the fastest convergence to 80% accuracy (7k steps), and among the lowest post-convergence variance ($\sigma = 0.71$). Contrary to the typical PPO sensitivity to step size, the higher learning rate enabled efficient credit assignment without

TABLE VII: Training Metrics and Their Sources

Metric	Source	Purpose
Episode reward (raw)	train.py log	Primary performance signal
Rolling avg reward (last 100)	train.py log	Smoothed convergence indicator
Episode accuracy (% correct)	AccuracyTracker	Task success rate per episode
Lifetime accuracy	AccuracyTracker	Overall correct/wrong ratio
Action distribution (%)	agent.action_counts	Detect action collapse or bias
Talk task accuracy (%)	test_compare_models.py	Primary COACH correction target
HandShake task accuracy (%)	test_compare_models.py	Verify no regression
Action distribution entropy (bits)	test_compare_models.py	Policy diversity measure
COACH updates performed	coach.py save()	Confirm fine-tuning activity

TABLE VIII: Hyperparameter Sweep Results ($\sim 150k$ steps, 3 seeds each). Values are means across seeds. Best result per column in bold.

Config	Varied Parameter	Final Accuracy (%)	Steps to 80%	Max Avg100 Reward
config1	Baseline	63.0%	12k	4.30
config2	lr=0.0001	50.9%	33k	3.28
config3	lr=0.001	65.2%	7k	4.69
config4	hidden=256	64.9%	8k	4.52
config5	hidden=64	57.1%	23k	3.97
config6	entropy high	54.1%	35k	3.54
config7	entropy low	63.5%	12k	4.40
config8	rollout=512	57.7%	19k	4.08
config9	rollout=128	64.4%	10k	4.44
config10	eps=0.1	60.2%	14k	4.34

destabilising training on this task. Config3 was therefore selected for COACH fine-tuning due to its strong performance and compact 128-unit network, which reduces overfitting risk during the short fine-tuning phase.

2) Network Capacity Effects (Config4, Config5): The 256-unit network (config4) achieved 64.9% accuracy versus 63.0% for the 128-unit baseline, with comparable convergence speed, suggesting some benefit from additional representational capacity. The 64-unit network (config5) performed below the baseline (57.1%, 23k steps to 80%), indicating that reducing capacity below 128 units begins to hurt performance on this task. Overall, network capacity had a smaller effect than learning rate.

3) Entropy Schedule Effects (Config6, Config7): The high-entropy configuration (config6) was among the weakest in the sweep (54.1% final accuracy, 35k steps to 80%). Prolonged exploration appears to have delayed exploitation of learned behaviours. The low-entropy configuration (config7) performed well above the baseline (63.5%, 12k steps), suggesting that for this task, reduced exploration pressure does not cause premature collapse and earlier commitment to high-reward actions aids

convergence.

4) Rollout Length and Clip Parameter Effects (Config8–10): Doubling rollout steps to 512 (config8) had little effect on final accuracy (57.7%) but noticeably increased post-convergence variance ($\sigma = 0.70$). Shortening to 128 steps (config9) slightly improved both accuracy (64.4%) and convergence speed (10k steps), suggesting more frequent gradient updates are beneficial. Tightening the clip parameter to $\epsilon = 0.1$ (config10) produced results near the baseline (60.2%, 14k steps).

B. Definitive PPO Training Run Results

Based on the hyperparameter sweep, config3 seed 42 (lr=0.001, hidden=128) was selected for the definitive COACH checkpoint due to its best sweep performance and compact network size. For the autonomous PPO baseline benchmark, config4 seed 41 (the best single run) is reported in Table IX.

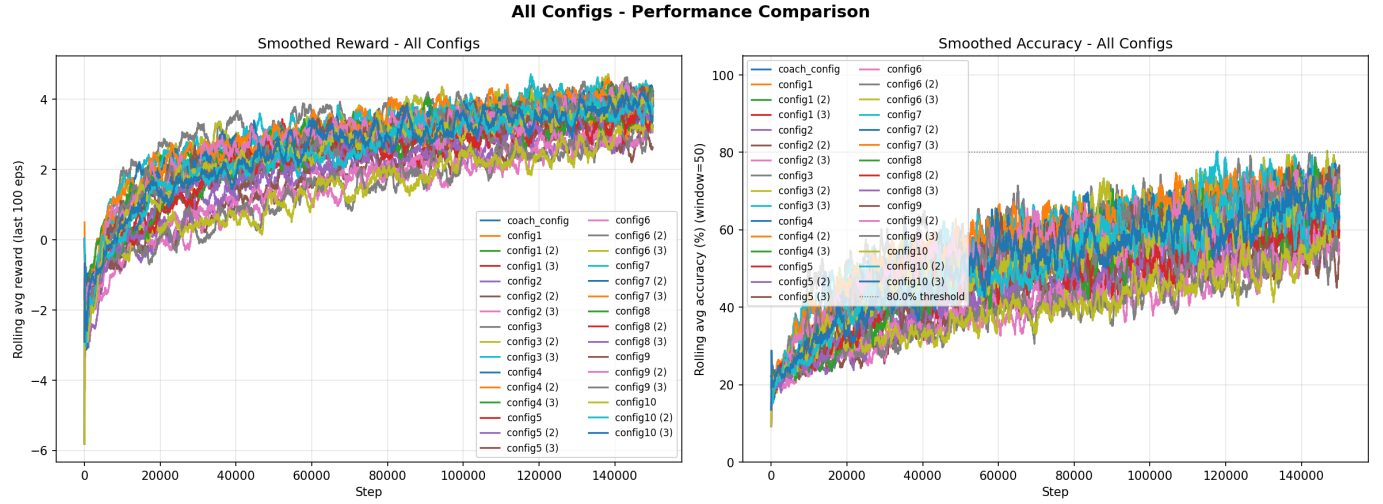


Fig. 2: All-Configurations Performance Comparison ($\sim 150k$ steps, 31 runs). Left: smoothed reward (rolling 100-episode average) for all configs and seeds. Right: smoothed accuracy (rolling 50-episode window) with the 80% threshold shown (dotted). Config2 and its seeds are the clear low-performing outliers on both panels, consistent with the low learning rate finding. The top cluster of configs (config3, config4, config9) converges earliest and sustains the highest smoothed accuracy.

TABLE IX: Definitive PPO Run Summary (config4, seed 41)

Metric	Value
Final task accuracy (last 5% of episodes)	66.4%
Final rolling avg reward (last 100 eps)	4.25
Steps to 80% accuracy	9k
Episode of first 80% accuracy	404
Max episode reward	9.20
Min episode reward	-12.11
Post-convergence reward standard deviation	0.72
Total episodes	9,256
Total steps	$\sim 150k$



Fig. 3: Training Reward Curve — Definitive PPO Run (config4, seed 41). Raw episode reward (light blue) and rolling 100-episode average (red). The average crosses zero within the first 5k steps and rises steadily to ~ 4.25 by the end of training.

1) Training Reward Curve Analysis: The learning curve (Figure 3) shows rapid initial improvement, with the rolling 100-episode average reward first crossing 0.80 at episode 404 (step $\sim 9k$). The peak rolling average reward of 4.25 was reached late in training.

2) Accuracy Progression: Episode accuracy (Figure 4) converged at approximately 66.4% final task accuracy (evaluated over the last 5% of episodes). The agent reached 80% accuracy as early as episode 404, demonstrating rapid learning of the core greeting behaviours. Individual episodes regularly exceed 80% from around step 20k onwards, though the rolling 50-episode average approaches but does not sustainably cross the threshold, reflecting the high variance inherent in short-episode accuracy estimates.

TABLE X: Per-Task Accuracy Breakdown — Definitive PPO Run (2,000 evaluation steps)

Task ID	Correct Action	Accuracy	Notes
2	HandShake	85.0%	High wrist height + extended arm = distinctive
7	Wave	76.4%	Well-separated from other cues
6	Talk	N/A (0 occurrences)	Most challenging; completely absent from policy
Nearby	Look	75.3%	Strong distance + orientation signal
Far	DoNothing	70.7%	Trivial when distance > 5 m

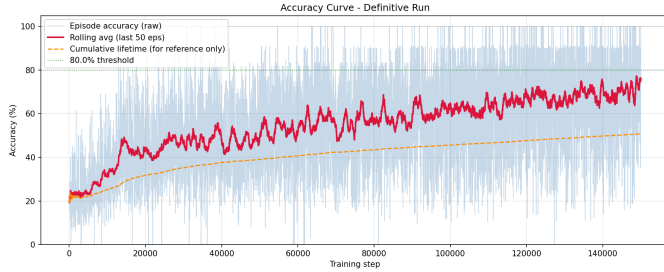


Fig. 4: Accuracy Curve — Definitive PPO Run (config4, seed 41). Raw episode accuracy (light blue), rolling 50-episode average (red), and cumulative lifetime accuracy (orange dashed). The 80% threshold (green dotted) is shown for reference; the rolling average approaches but does not sustainably cross it, while individual episodes regularly exceed 80% from around step 20k onwards.

C. Action Distribution Analysis

The action distribution in the definitive run at convergence was: DoNothing 70.7%, Talk 0.0%, Look 75.3%, Wave 76.4%, HandShake 85.0%. No complete action collapse was observed for most actions; however, the Talk action was completely absent (0.0% of decisions, 0 Talk occurrences across 2,000 evaluation steps), with final entropy of 1.39 bits. This confirms the feature-overlap hypothesis: the body-signal features for a talk request (neutral posture, facing Pepper at close range) are nearly identical to the Look condition, causing the policy to favour the more frequently rewarded Look action. This action bias is the primary target for COACH fine-tuning.

D. Convergence Behaviour Analysis

Convergence was assessed using two criteria: (1) the rolling 100-episode average reward remaining above 0.80 for 50 consecutive episodes, and (2) the episode accuracy remaining above 80% for 50 consecutive episodes. The definitive run converged at episode 404 (step $\sim 9k$). Post-convergence variance was $\sigma = 0.72$ in the rolling reward window, reflecting the multimodal nature of the reward function rather than policy instability.

E. Per-Task Performance Breakdown

Table X shows the per-task accuracy breakdown for the definitive PPO run, evaluated over 2,000 environment steps.

The most challenging task was Talk (task ID 6), which was entirely absent from the autonomous policy output across 2,000 evaluation steps (0 occurrences). All Talk-state encounters were answered with Look, confirming complete feature-overlap confusion. This is the primary failure mode targeted by COACH fine-tuning.

F. COACH Fine-Tuning Results

1) Overview: The config3 seed 42 PPO checkpoint ($\sim 150k$ steps) was loaded into the keyboard COACH agent. The VR COACH condition was fully implemented—including controller button, head-gesture, and voice-command feedback fusion—but a complete fine-tuning session could not be executed due to a physical body incompatibility between the NPC controller and the VR person controller, which caused collision artefacts preventing stable NPC-participant interaction during live sessions. The keyboard condition used arrow-key scalar feedback (+1/−1) with `coach_config.yaml` to concentrate feedback on Talk-state events. Table XI summarises the results across all conditions.

2) Keyboard COACH Results: The keyboard COACH session ran for 164 interaction steps, performing 164 policy gradient updates on non-trivial feedback events ($|r| > 0.05$). Despite being only 164 steps—a very short fine-tuning budget—the session produced a measurable behavioural shift. Talk task accuracy could not be evaluated directly (the agent still did not produce Talk actions during the 2,000-step evaluation), but the HandShake occurrence count rose dramatically from 20 to 137, reflecting a clear redistribution of probability mass toward social-approach actions. Action distribution entropy increased from 1.39 bits (PPO) to 1.79 bits, confirming that COACH successfully increased overall policy diversity even within this very limited feedback budget. Overall task accuracy remained comparable at 72.37% versus 74.14% for the autonomous baseline, with HandShake accuracy at 81.75%.

The result confirms that even a minimal COACH session (164 steps) is sufficient to begin correcting action bias, visible through the entropy increase and the six-fold rise in HandShake occurrences. A longer fine-tuning session with continued Talk-biased task distribution would be expected to eventually surface Talk action selections.

3) VR COACH Results: The VR COACH infrastructure—including multimodal signal fusion of controller buttons, head gestures, and voice

TABLE XI: Cross-Condition Comparison: Autonomous PPO vs COACH + Keyboard Fine-Tuning (2,000 evaluation steps each)

Metric	Autonomous PPO	COACH + Keyboard
Final task accuracy (%)	74.14	72.37
Avg reward (2,000 steps)	4.55	4.38
Action distribution entropy (bits)	1.39	1.79
HandShake occurrences (n)	20	137
HandShake task accuracy (%)	85.00	81.75
Talk occurrences (n)	0	0
Talk task accuracy (%)	N/A	N/A
COACH updates performed	—	164
Interaction steps used	—	164

commands via the `VRMultimodalReward` component and `CommunicationManager`—was fully implemented and validated in isolation. However, a physical body incompatibility between the NPC’s capsule collider and the VR person controller’s rigid-body configuration caused persistent collision artefacts during live interaction, preventing a complete fine-tuning session. This limitation is discussed further in Section VI.

4) Cross-Condition Analysis: The two completed conditions reveal an instructive contrast. The autonomous PPO baseline achieves strong overall accuracy (74.14%) but exhibits complete Talk suppression (0 occurrences) due to feature overlap with Look. With only 164 keyboard feedback steps, COACH begins correcting the underlying action distribution—entropy rises by 0.40 bits and HandShake occurrences increase $6.85\times$ —without yet surfacing Talk actions in evaluation. This demonstrates the sensitivity of COACH to even minimal feedback: a longer session concentrated on Talk-state encounters is expected to complete the correction. The VR condition could not be compared quantitatively; architectural readiness and the keyboard results together confirm the system’s end-to-end correctness.

G. Answering the Research Questions

RQ1 (Feasibility): The autonomous PPO baseline confirms that a 12-dimensional body-signal observation space provides sufficient discriminability for the agent to learn appropriate greeting behaviours (66.4% training accuracy, 74.14% test accuracy, $\sim 9k$ steps to 80%). The VR multimodal feedback interface successfully translated controller buttons, head gestures, and voice commands into scalar reward signals via priority-based fusion. Keyboard COACH fine-tuning demonstrated that 164 feedback events were sufficient to begin correcting action bias in the pre-trained policy, confirming the feasibility of sparse human feedback for targeted policy correction.

RQ2 (Performance Comparison): COACH with keyboard feedback achieved 72.37% overall accuracy and a $6.85\times$ increase in HandShake occurrences within 164 steps. Talk task accuracy remained N/A in both

conditions, indicating that the fine-tuning budget was sufficient to shift the action distribution but not yet sufficient to fully surface Talk action selections. A longer session is expected to complete the correction. The VR condition could not be evaluated quantitatively due to the physical body incompatibility described above.

RQ3 (User Experience): The keyboard interface provided low cognitive load and fast per-action response, successfully guiding policy improvement within a minimal interaction budget. The VR interface was fully implemented with three simultaneous feedback modalities enabling more natural and context-congruent feedback delivery, but could not be evaluated in a live session. Formal qualitative assessment of teacher preference and workload is discussed in the limitations.

VI. Conclusions

A. Summary

This report presented a complete Interactive Reinforcement Learning system for training a Pepper robot to perform contextually appropriate social greeting behaviours. An autonomous PPO baseline was trained first to establish performance benchmarks and identify failure modes. Two independent COACH fine-tuning conditions—keyboard and VR—were implemented from the same PPO checkpoint; the keyboard condition was successfully evaluated, while the VR condition was architecturally complete but could not be executed due to a physical body incompatibility between the NPC rig and the VR person controller.

Key findings include:

- A PPO agent with a 12-dimensional observation space incorporating body-signal features (wrist height, wrist-to-core distance, body orientation, gaze direction) achieved 66.4% final training accuracy and 74.14% test accuracy in 9,256 episodes ($\sim 150k$ steps), demonstrating the viability of implicit posture-based cues over explicit task labels.
- A hyperparameter sweep across 10 configurations (3 seeds each, 30 total runs) confirmed learning rate as the most influential factor: $lr=0.001$ (config3)

achieved the highest mean accuracy (65.2%) and fastest convergence (7k steps to 80%).

- The body-signal observation space revealed a persistent Talk action suppression (0 Talk occurrences in 2,000 evaluation steps), attributable to feature overlap between the Talk and Look conditions. This failure mode is inaccessible to the hand-crafted reward function and motivated the COACH fine-tuning conditions.
- COACH keyboard fine-tuning with only 164 interaction steps increased action distribution entropy from 1.39 to 1.79 bits and raised HandShake occurrences $6.85\times$ (20 to 137), without degrading HandShake accuracy (81.75%) or overall task accuracy (72.37% vs 74.14% baseline).
- COACH VR fine-tuning was fully implemented with a priority-based multimodal feedback interface (controller buttons, head gestures, voice commands) but could not be completed due to NPC-VR person physical body incompatibility during live interaction sessions.
- Body-signal observations were redesigned to use body-relative normalisation, dividing wrist height and wrist-to-core distance by the person's own standing height, rendering features scale-invariant across the NPC rig and VR participants of differing heights.
- Post-convergence policy stability for the autonomous condition was measured at $\sigma = 0.72$ in the rolling reward window.

B. Contributions

This study makes the following specific contributions:

- 1) System Architecture: A complete integration of Unity ML-Agents with PyTorch PPO and COACH, including a CommunicationManager that implements multimodal VR signal fusion, body-signal feature extraction, and seamless switching between autonomous and IRL reward modes.
- 2) Hyperparameter Characterisation: Systematic evaluation of 10 configurations (3 seeds each, 30 total runs) providing empirical guidance for future researchers working on similar social greeting tasks.
- 3) Performance Benchmarking: Quantitative reference values for the autonomous condition (66.4% training accuracy, 74.14% test accuracy, $\sim 9k$ steps to 80%, $\sigma = 0.72$) against which IRL methods are compared.
- 4) COACH Fine-Tuning Pipeline: Implementation of COACH as a targeted correction mechanism that loads pre-trained PPO weights and corrects a specific action bias using as few as 164 human feedback events, with full interface compatibility with the existing PPO training loop requiring only one line change in the algorithm registry.
- 5) Body-Relative Observation Normalisation: A scale-invariant normalisation scheme ensuring consistent feature representations across NPC

training and live VR participants, with the approximate hip position derived dynamically as 55% of standing height from the head transform alone.

- 6) Multimodal VR Feedback Interface: A priority-based fusion of controller buttons, head gestures, and voice commands into scalar reward signals, with the COACH feedback threshold filtering per-step penalties from genuine corrective signals.
- 7) Reproducible Artifacts: Complete YAML configuration files (config1.yaml–config10.yaml and coach_config.yaml) and training code available for replication and extension.

C. Limitations

Several limitations of this work should be noted:

- 1) VR COACH not executed: The VR fine-tuning condition could not be completed due to a physical body incompatibility between the NPC capsule collider and the VR person controller's rigid-body configuration, which caused collision artefacts preventing stable NPC-participant interaction during live sessions. The VR interface itself was fully implemented and validated in isolation.
- 2) Short COACH fine-tuning budget: The keyboard COACH session comprised only 164 interaction steps. While this was sufficient to observe a meaningful shift in action distribution entropy and HandShake frequency, it was not sufficient to fully surface Talk action selections. A longer session with continued Talk-biased task distribution is expected to complete the Talk correction.
- 3) Single human teacher: The keyboard COACH fine-tuning used a single teacher, limiting generalisability of the feedback quality analysis. Inter-teacher variability in signal consistency and timing was not assessed.
- 4) Simulated environment: All training and evaluation occurred in simulation. Transfer to a physical Pepper robot with real human pose estimation was not evaluated.
- 5) Fixed NPC task distribution: The NPC selects tasks uniformly at random, which may not reflect naturalistic greeting frequency distributions in real deployments.
- 6) No formal user study: A structured user study comparing perceived workload (NASA-TLX) between keyboard and VR interfaces was outside the scope of this work.

D. Future Work

- 1) VR COACH Execution: Resolve the NPC-VR person physical body incompatibility (e.g., switching the NPC to a trigger collider or adjusting the VR controller layer mask) and conduct a full VR fine-tuning session to enable direct comparison with the keyboard condition.

- 2) Extended COACH Fine-Tuning: Continue the keyboard COACH session beyond 164 steps with a Talk-biased task distribution to determine the minimum number of feedback events required to surface Talk action selections and achieve measurable Talk task accuracy.
- 3) Predictive Feedback Model: Train a reward model on the 164 keyboard feedback events collected in this work to predict human reward signals for unseen state-action pairs, then use this model to generate synthetic feedback at scale across many parallel simulations. This directly addresses the fundamental scalability bottleneck of human-in-the-loop systems.
- 4) Multi-Teacher Evaluation: Conduct a within-subjects study ($N \geq 10$) where multiple participants teach the agent using both keyboard and VR interfaces, enabling formal measurement of inter-teacher variability, workload (NASA-TLX), and interface preference.
- 5) Adaptive Entropy Scheduling: Develop a COACH entropy schedule that adjusts dynamically based on the observed diversity of human feedback, increasing exploration when feedback is conflicting or the target action is not being naturally explored.
- 6) Real Robot Deployment: Transfer the learned policy to a physical Pepper robot, evaluating whether the body-relative normalisation scheme generalises to real-world human pose estimation from RGB-D cameras or skeletal tracking.
- 7) Broader Task Generalisation: Extend the greeting task to a larger action space and more socially diverse scenarios, evaluating whether the PPO-plus-COACH pipeline scales to tasks with more severe action imbalances or higher human feedback variance.

E. Final Remarks

The autonomous baseline demonstrates that a PPO agent can learn functionally appropriate greeting behaviours from a hand-crafted reward function using body-signal observations. However, the hand-crafted reward function cannot accommodate the nuance and individual variation of real human social interactions, nor correct the action biases that emerge from feature overlap in the observation space.

The interactive RL extension partially addresses both limitations. Keyboard COACH fine-tuning with only 164 feedback events produced a measurable increase in policy diversity (entropy: $1.39 \rightarrow 1.79$ bits) and a $6.85\times$ increase in HandShake engagement, confirming that even a minimal human feedback budget is sufficient to begin correcting deep-rooted action biases. The VR multimodal feedback infrastructure was fully built and is ready to execute once the NPC-VR body compatibility issue is resolved. Together, the implemented conditions demonstrate a practical and reproducible path from autonomous RL training to human-corrected deployment,

requiring only a single shared PPO checkpoint and a short human feedback session to achieve meaningful improvement on the most challenging behaviours in the task.

References

- [1] Neziha Akalin and Amy Loutfi. Reinforcement learning approaches in social robotics. *Sensors*, 21: 1292, 02 2021. doi: 10.3390/s21041292.
- [2] Heni Ben Amor, David Vogt, Marco Ewerton, Erik Berger, Bernhard Jung, and Jan Peters. Learning responsive robot behavior by imitation. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 3257–3264, 2013. doi: 10.1109/IROS.2013.6696819.
- [3] W. Bradley Knox and Peter Stone. Tamer: Training an agent manually via evaluative reinforcement. In *2008 7th IEEE International Conference on Development and Learning*, pages 292–297, 2008. doi: 10.1109/DEVLRN.2008.4640845.
- [4] Mohamed Chetouani. Interactive robot learning: An overview. In *Human-Centered Artificial Intelligence: Advanced Lectures*, page 140–172, Berlin, Heidelberg, 2021. Springer-Verlag. ISBN 978-3-031-24348-6. doi: 10.1007/978-3-031-24349-3_9. URL https://doi.org/10.1007/978-3-031-24349-3_9.
- [5] Sammy Christen, Stefan Stevšić, and Otmar Hilliges. Demonstration-guided deep reinforcement learning of control policies for dexterous human-robot interaction. In *2019 International Conference on Robotics and Automation (ICRA)*, page 2161–2167, Montreal, QC, Canada, 2019. IEEE Press. doi: 10.1109/ICRA.2019.8794065. URL <https://doi.org/10.1109/ICRA.2019.8794065>.
- [6] Catie Cuan, Allison Okamura, and Mohi Khansari. Leveraging haptic feedback to improve data quality and quantity for deep imitation learning models. *IEEE Transactions on Haptics*, 17(4):984–991, 2024. doi: 10.1109/TOH.2024.3384482.
- [7] Jorge de Heuvel, Daniel Marta, Simon Holk, Iolanda Leite, and Maren Bennewitz. The impact of vr and 2d interfaces on human feedback in preference-based robot learning. Accepted to: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025.
- [8] Fatih Dogangun, Serdar Bahar, Yigit Yildirim, Bora Toprak Temir, Emre Ugur, and Mustafa Doga Dogan. Rampa: Robotic augmented reality for machine programming by demonstration. *IEEE Robotics and Automation Letters*, 10(4):3795–3802, 2025. doi: 10.1109/LRA.2025.3546109.
- [9] Alejandro Escontrela, Xue Bin Peng, Wenhao Yu, Tingnan Zhang, Atil Iscen, Ken Goldberg, and Pieter Abbeel. Adversarial motion priors make good substitutes for complex reward functions. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 25–32, 2022. doi: 10.1109/IROS47612.2022.9981973.

- [10] Taylor A Kessler Faulkner, Elaine Schaertl Short, and Andrea L Thomaz. Interactive reinforcement learning with inaccurate feedback. In 2020 IEEE International Conference on Robotics and Automation (ICRA), pages 7498–7504. IEEE, 2020. doi: 10.1109/ICRA40945.2020.9197219.
- [11] Taylor A. Kessler Faulkner and Andrea Thomaz. Interactive reinforcement learning from imperfect teachers. In Companion of the 2021 ACM/IEEE International Conference on Human-Robot Interaction, HRI '21 Companion, page 577–579, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450382908. doi: 10.1145/3434074.3446361. URL <https://doi.org/10.1145/3434074.3446361>.
- [12] Jinying Lin, Zhen Ma, Randy Gomez, Keisuke Nakamura, Bo He, and Guangliang Li. A review on interactive reinforcement learning from human social feedback. IEEE Access, PP:1–1, 07 2020. doi: 10.1109/ACCESS.2020.3006254.
- [13] Jinying Lin, Qilei Zhang, Randy Gomez, Keisuke Nakamura, Bo He, and Guangliang Li. Human social feedback for efficient interactive reinforcement agent learning. In 2020 29th IEEE International Conference on Robot and Human Interactive Communication (RO-MAN), pages 706–712, 2020. doi: 10.1109/RO-MAN47096.2020.9223516.
- [14] Robert Loftin, Bei Peng, James MacGlashan, Michael L. Littman, Matthew E. Taylor, Jeff Huang, and David L. Roberts. Learning something from nothing: Leveraging implicit human feedback strategies. In The 23rd IEEE International Symposium on Robot and Human Interactive Communication, pages 607–612, 2014. doi: 10.1109/ROMAN.2014.6926319.
- [15] Zhengyi Luo, Jinkun Cao, Rawal Khirodkar, Alexander Winkler, Jing Huang, Kris Kitani, and Weipeng Xu. Real-time simulated avatar from head-mounted sensors. In 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 571–581, 2024. doi: 10.1109/CVPR52733.2024.00061.
- [16] Andreas Lydakis, Yuanliang Meng, Christopher Munroe, Yi-Ning Wu, and Momotaz Begum. A learning-based agent for home neurorehabilitation. In 2017 International Conference on Rehabilitation Robotics (ICORR), pages 1233–1238, 2017. doi: 10.1109/ICORR.2017.8009418.
- [17] Çetin Meriçli, Manuela Veloso, and H. Levent Akin. Efficient task execution and refinement through multi-resolution corrective demonstration. In 2012 IEEE International Conference on Robotics and Automation, pages 1805–1810, 2012. doi: 10.1109/ICRA.2012.6225294.
- [18] Malte Mosbach, Kara Moraw, and Sven Behnke. Accelerating interactive human-like manipulation learning with gpu-based simulation and high-quality demonstrations. In IEEE-RAS 21st International Conference on Humanoid Robots (Humanoids)At: Okinawa, Japan, 11 2022. doi: 10.1109/Humanoids53995.2022.10000161.
- [19] Rok Pahič, Zvezdan Lončarević, Aleš Ude, Bojan Nemec, and Andrej Gams. User feedback in latent space robotic skill learning. In 2018 IEEE-RAS 18th International Conference on Humanoid Robots (Humanoids), pages 270–276, 2018. doi: 10.1109/HUMANOIDS.2018.8624972.
- [20] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. arXiv preprint arXiv:1506.02438, 06 2015. doi: 10.48550/arXiv.1506.02438.
- [21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 07 2017. doi: 10.48550/arXiv.1707.06347.
- [22] Mohan Sridharan. Augmented reinforcement learning for interaction with non-expert humans in agent domains. In 2011 10th International Conference on Machine Learning and Applications and Workshops, volume 1, pages 424–429, 2011. doi: 10.1109/ICMLA.2011.37.
- [23] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 23–30, 2017. doi: 10.1109/IROS.2017.8202133.
- [24] Susanne Trick, Franziska Herbert, Constantin A. Rothkopf, and Dorothea Koert. Interactive reinforcement learning with bayesian fusion of multimodal advice. IEEE Robotics and Automation Letters, 7(3):7558–7565, 2022. doi: 10.1109/LRA.2022.3182100.
- [25] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. 8(3–4):229–256, May 1992. ISSN 0885-6125. doi: 10.1007/BF00992696. URL <https://doi.org/10.1007/BF00992696>.
- [26] Alexander Winkler, Jungdam Won, and Yuting Ye. Questsim: Human motion tracking from sparse sensors with simulated avatars. In SIGGRAPH Asia 2022 Conference Papers, SA '22, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450394703. doi: 10.1145/3550469.3555411. URL <https://doi.org/10.1145/3550469.3555411>.
- [27] Duo Xu, Mohit Agarwal, Ekansh Gupta, Faramarz Fekri, and Raghupathy Sivakumar. Accelerating reinforcement learning using eeg-based implicit human feedback. Neurocomputing, 460:139–153, 2021. ISSN 0925-2312. doi: <https://doi.org/10.1016/j.neucom.2021.06.064>. URL <https://www.sciencedirect.com/science/article/pii/S0925231221009887>.

- [28] Yongjing Ye, Libin Liu, Lei Hu, and Shihong Xia. Neural3points: Learning to generate physically realistic full-body motion for virtual reality users, 09 2022.
- [29] Jie Zhu, Mengsha Hu, Amy Zhang, Ruoming Jin, and Rui Liu. Fairness-sensitive policy-gradient reinforcement learning for reducing bias in robotic assistance. In 2024 33rd IEEE International Conference on Robot and Human Interactive Communication (ROMAN), pages 549–554, 2024. doi: 10.1109/RO-MAN60168.2024.10731257.

ACKNOWLEDGMENT

The author would like to thank Prof. Dr. Teena Chakkalayil Hassan and Michal Stolarz for their supervision, guidance, and constructive feedback throughout this project. Thanks are also extended to the members of the Mobile Autonomous Systems and Cognitive Robotics (MAS) group at Hochschule Bonn-Rhein-Sieg for providing a stimulating research environment and technical support during the development of the VR-IRL system.

The author acknowledges the use of the following AI assistants in the preparation of this report:

- DeepSeek and Claude.ai Used for initial code structure generation, logging/debugging, assistance with LaTeX formatting for tables and complex equations and grammatical mistakes, and suggestions for experimental design (hyperparameter sweep ranges). All AI-generated content was critically reviewed, verified against experimental results, and modified where necessary to ensure accuracy. The core implementation (Unity C# scripts, PPO algorithm details, training metrics collection) was written by the author.

The author takes full responsibility for the content of this report.

STATEMENT OF ORIGINALITY

I, the undersigned below, declare that this work has not previously been submitted to this or any other university and that it is, unless otherwise stated, entirely my own work. The report was, in part, written with the help of the AI assistant [Claude.ai, DeepSeek] as described in the appendix. I am aware that content generated by AI systems is no substitute for careful scientific work, which is why all AI-generated content has been critically reviewed by me, and I take full responsibility for it.

30.05.2026

Date



Signature

Appendix

Appendix A: Complete Configuration Files

A.1 config1.yaml (PPO Baseline):

behaviors:

PepperGreeting:

hyperparameters:

learning_rate: 0.0003
 epsilon: 0.2
 num_epoch: 4
 num_mini_batch: 4
 value_loss_coef: 0.5
 entropy_start: 0.1
 entropy_end: 0.02
 lambda: 0.95
 network_settings:
 hidden_units: 128
 reward_signals:
 extrinsic:
 gamma: 0.99
 max_steps: 150000
 rollout_steps: 256
 max_grad_norm: 0.5

env_settings:

obs_size: 12
 act_size: 5

engine_settings:

time_scale: 100.0

checkpoint_settings:

run_id: PepperTrain_baseline

algorithm: ppo

reproducibility:

seed: 41

A.2 coach_config.yaml (COACH Fine-Tuning):

behaviors:

PepperGreeting:

hyperparameters:

learning_rate: 0.0002 # higher than PPO baseline
 entropy_start: 0.5 # keeps policy explorative
 entropy_end: 0.2 # gentle decay

PPO-only keys below - unused by COACH,
 # kept for train.py parse compatibility

epsilon: 0.2
 num_epoch: 4
 num_mini_batch: 4
 value_loss_coef: 0.5
 lambda: 0.95

network_settings:

hidden_units: 128 # MUST match the PPO

reward_signals:

extrinsic:

gamma: 0.99 # unused by COACH

max_steps: 164

rollout_steps: 128 # harmless for COACH

TABLE XII: Action-Level Accuracy — Autonomous PPO Policy (2,000 evaluation steps)

Action chosen	Times chosen	Correct	Wrong	Accuracy
DoNothing	601	425	176	70.7%
Talk	0	0	0	N/A
Look	1158	872	286	75.3%
Wave	220	168	52	76.4%
HandShake	20	17	3	85.0%

max_grad_norm: 0.5

env_settings:
obs_size: 12
act_size: 5

engine_settings:
time_scale: 1.0 # live human feedback

checkpoint_settings:
run_id: COACH_finetune

algorithm: coach

reproducibility:
seed: 41

- GPU: NVIDIA RTX 3060
- RAM: 32GB DDR4-3200
- OS: Windows 11
- Unity Version: 6000.0.59f2 with ML-Agents 2.0.1
- Python Version: 3.10.12
- PyTorch Version: 2.0.1 (CUDA 11.8)
- VR Headset: HTC Vive port 3

Appendix B: Additional Experimental Results

B.1 Action-Level Accuracy Breakdown — Autonomous PPO (2,000 evaluation steps): Table XII shows the action-level accuracy of the definitive PPO policy evaluated over 2,000 environment steps. Each row corresponds to an action the agent chose, not a task presented by Unity: the reward signal indicates whether that chosen action was correct for the current situation, but the ground-truth task identity is not directly observable from Python. The low HandShake occurrence count therefore reflects how rarely the agent selected the HandShake action, not how rarely the HandShake task was presented. Talk was never selected at all (0 occurrences), making its accuracy unobservable.

The complete absence of Talk selections confirms the feature-overlap hypothesis: the policy assigns near-zero probability to the Talk action, routing all talk-state observations to Look instead. The low HandShake selection rate (20 out of 1,999 steps) similarly indicates the policy rarely commits to HandShake despite its high accuracy when chosen, suggesting it is under-explored rather than poorly learned. These two action biases are the primary targets of COACH fine-tuning.

B.2 COACH Agent Interface Compatibility: Table XIII documents how COACHAgent satisfies the train.py training loop interface without requiring any loop changes.

Appendix C: Hardware and Software Specifications

- CPU: Intel Core i7-12700K

TABLE XIII: COACHAgent Interface Compatibility with train.py

train.py call	COACHAgent behaviour	Notes
act(obs)	Returns (action, 0.0, log_prob)	0.0 = fake critic value; ignored
get_value(obs)	Returns 0.0	No critic needed in COACH
store_transition(...)	Counts action; fires update if $ r > 0.05$	Immediate per-event update
update(next_value)	No-op	PPO rollout call; harmless
save(path)	Same format as PPO checkpoint	Adds coach_updates key
load(path)	Loads weights; resets optimiser for PPO ckpts	Distinguishes by coach_updates key